Submodule 33.09: Optimal Cluster Selection: Elbow Method & Silhouette Score (Hinglish)

Topic Overview & Business Constraints

Optimal number of clusters (K) decide karna machine learning accuracy aur business feasibility ke beech ka balance hota hai:

- E-commerce marketing team ke liye 20 ya 30 clusters manage karna impossible hota hai.
- Isliye hum $K \in [1, 10]$ ki range mein test karte hain taaki 3 se 6 solid personas mil sakein.

---

1. Optimal K Dhundhne ke Do Methods

A. Elbow Method (WCSS / Inertia)

Har cluster ke andar data points ke centroid se square distance ka sum:

$$\text{WCSS} = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

Jaise-jaise K badhta hai, WCSS drop hota hai. Jis point ke baad WCSS ke drop hone ki speed slow ho jati hai, use "Elbow point" kehte hain.

B. `KneeLocator` se Objective Elbow Detection

Aankhon se guess karne ki jagah `kneed.KneeLocator` mathematically curve ke maximum convexity point ko detect karta hai:

$$\text{optimal_k} = 4$$

C. Silhouette Score

Silhouette score measure karta hai ki data points apne cluster ke kitne pass hain ($a(i)$) aur nearest neighbor cluster se kitne door hain ($b(i)$):

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

---

2. Python Code Implementation (Dual-Axis Plot)

Reference: smartcart.ipynb (Cells 46 - 53).

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from kneed import KneeLocator
import matplotlib.pyplot as plt

# 1. WCSS calculate karna (K = 1 to 10)
wcss = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)

# Automatic Elbow Find karna
knee = KneeLocator(range(1, 11), wcss, curve="convex", direction="decreasing")
optimal_k = knee.elbow
print("Best K from KneeLocator:", optimal_k) # Output: 4

# 2. Silhouette Score calculate karna (K = 2 to 10)
scores = []
for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(X_pca)
    scores.append(silhouette_score(X_pca, labels))

# 3. Dual-Axis Plot (ax1: WCSS, ax2: Silhouette Score)
k_range = range(2, 11)
fig, ax1 = plt.subplots(figsize=(8, 6))

ax1.plot(k_range, wcss[1:], marker="o", color="blue", label="WCSS")
ax1.set_xlabel("Number of Clusters (K)")
ax1.set_ylabel("WCSS (Blue)", color="blue")

ax2 = ax1.twinx()
ax2.plot(k_range, scores, marker="x", color="red", linestyle="--", label="Silhouet
ax2.set_ylabel("Silhouette Score (Red)", color="red")

plt.title("Combined Elbow & Silhouette Plot")
plt.show()
```

---

3. Decision: $K = 4$

- $K = 4$ par WCSS curve sharp elbow banata hai.
- Silhouette score robust balance maintain karta hai.
- 4 customer segments commercial marketing ke liye perfectly manageable hain.